minimises the attack vectors that your image is exposed to. And when we say all the dependencies, we mean all. There's no point in containerising your project if you can't flick in on post migration. If you're required to reinstall a few dependencies each time you move your project then it's not exactly containerised. Repeatability is the name of the game.

Since we're looking at the Windows environment, there are a few commands that you need to be aware of for PowerShell.

- **Add-WindowsFeature** - Allows you to install Windows features in your image
- **Get-WindowsFeature** - Shows you all the features that are available for install

You should always check to see which features are pre-installed and what version they're running.

To download packages from the internet, you'll have to run the Invoke-WebRequest cmdlet. This cmdlet sends HTTP and HTTPS requests and can collect links, images and packages. We'll store the version of the software and the checksum hash as environment variables and use Get-FileHash to verify the hash of the downloaded file with what's stored on the official repository.

Let's say you want to install the latest Node.js package into your image. You can use:

```
ENV NODE_VERSION="15.10.0" `
        NODE_SHA256="351ef617d
ea3ec93819ebb92435b82febaa1de
393bdb442dcc129ac76a03014c"

WORKDIR C:/node

RUN Invoke-WebRequest -Out-
File node.exe "https://
nodejs.org/dist/v$($env:NODE_
VERSION)/win-x64/node.exe"
-UseBasicParsing; `
        if ((Get-FileHash
node.exe -Algorithm sha256).
Hash -ne $env:NODE_SHA256)
{exit 1} ;
```

By pulling the Node.js version and the hash as separate environment variables, you can make it easy to rebuild the Dockerfile. Once the command has run, you can see if Node.js is properly installed in the specified directory. In this case, it would be C:/node.

SETTING UP ENTRYPOINTS

When an image is run in Docker, it starts whichever process is marked under the CMD or ENTRYPOINT instruction in the Dockerfile. The difference between CMD and ENTRYPOINT is that CMD is best used when you need a default command that can be overridden by the user. Whereas ENTRYPOINT is used if you want to run a specific executable. Here's an example of a Dockerfile for a simple Go application.

```
FROM golang:1.11-alpine AS
build
#Install required tools
RUN apk add --no-cache git
RUN go get github.com/golang/
dep/cmd/dep
#List project dependencies
with Gopkg,toml and Gopkg.
lock
COPY Gopkg.lock Gopkg.toml /
go/src/project/
WORKDIR /go/src/project/
#Install depdendencies
RUN dep ensure -vendor-only
#Copy the entire project and
build it
COPY . /go/src/project/
RUN go build -o /bin/project
#Specify the entrypoint
within the newly created
image
FROM scratch
COPY --from=build /bin/pro-
ject /bin/project
ENTRYPOINT ["/bin/project"]
CMD ["--help"]
```

As you can see, the process is similar to what you would do when using Linux but with Windows, there are a few different steps. For .NET web apps, you should be serving your app using the w3wp.exe process which runs in the IIS Windows service. You then use the ServiceMonitor.exe tool as an entrypoint since it checks to see if services are running and can raise a flag if there are any failures.

You can also write a PowerShell script to monitor IIS and add a layer of redundancy, if needed.

ADD HEALTHCHECKS

This one is obvious. You want to monitor your application within the Docker container to see if they're running healthy. HEALTHCHECK is a command that lets you do just that. Let's say you have an ASP .NET web app running and you need to check if everything is well. You can use the following lines in the Dockerfile to do that.

```
HEALTHCHECK --interval=5s `
 CMD powershell -command `
        try { `
        $response = iwr
http://localhost/diagnostics
-UseBasicParsing; `
        if ($response.Sta-
tusCode -eq 200) { return 0}
`
        else {return 1}; `
        } catch { return 1 }
```

Here, we're running HEALTHCHECK at an interval of every 5 seconds. If you don't specify the interval, then Docker will run it at a 30 second interval. What we're basically doing here is querying a known URL and checking if the status returned in 200 or not. If the response is 200, then the command exits with a code of 1, which tells us that the HEALTHCHECK ran fine. If the URL response is not 200, then the exit code will be 0 and we know that there's something wrong with our application.

THAT'S IT

The process is fairly simple, if you look at it. There are plenty of base images and derived images available than what we had a couple of years ago. Even though Windows isn't the favoured environment for developers, it's good to see that the provisions are present. You'll have to update your project to work with the .NET 4.5 image or whatever is current as older images are not maintained.